

ĐẠI HỌC QUỐC GIA TP HCM

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

---

## Báo cáo Project 03

Xv6 File System

---

Môn học: Hệ Điều Hành

*Sinh viên thực hiện:*

Phạm Phú Hòa (23122030)

Trần Tà Quang Minh (23122042)

Trần Chí Nguyên (23122044)

*Giáo viên hướng dẫn:*

Lê Giang Thanh



## Mục lục

|          |                                       |          |
|----------|---------------------------------------|----------|
| <b>1</b> | <b>Tổng quan</b>                      | <b>2</b> |
| <b>2</b> | <b>Bảng phân công công việc</b>       | <b>2</b> |
| <b>3</b> | <b>Chi tiết thực hiện</b>             | <b>2</b> |
| 3.1      | Large files - Mở rộng kích thước file | 2        |
| 3.1.1    | Phân tích vấn đề                      | 2        |
| 3.1.2    | Cài đặt chi tiết                      | 3        |
| 3.1.3    | Kết quả kiểm thử Large files          | 3        |
| 3.2      | Symbolic links - Liên kết mềm         | 4        |
| 3.2.1    | Phân tích yêu cầu                     | 4        |
| 3.2.2    | Cài đặt chi tiết                      | 4        |
| 3.2.3    | Kết quả kiểm thử Symbolic links       | 4        |
| 3.3      | Kết quả tổng hợp                      | 5        |
| 3.3.1    | Điểm số make grade                    | 5        |
| <b>4</b> | <b>Tổng kết</b>                       | <b>5</b> |
| 4.1      | Những gì đã học được                  | 6        |
| 4.2      | Khó khăn gặp phải                     | 6        |

## 1 Tổng quan

Phần bài tập này tập trung vào việc mở rộng và cải tiến hệ thống file của xv6. Nhóm thực hiện hai yêu cầu chính: hỗ trợ file có kích thước lớn thông qua cơ chế doubly-indirect block và cài đặt symbolic links (soft links) để liên kết file theo đường dẫn.

Các bài tập đã hoàn thành:

1. Mở rộng kích thước file tối đa (Large files) - Hỗ trợ file lên đến 65803 blocks.
2. Cài đặt symbolic links - Tạo liên kết mềm giữa các file trong hệ thống.

## 2 Bảng phân công công việc

| Thành viên                     | Công việc                                                                                                                                                                                                                     | Tỷ lệ |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Phạm Phú Hòa<br>23122030       | <ul style="list-style-type: none"><li>• Nghiên cứu cấu trúc inode và cơ chế block mapping</li><li>• Thiết kế và cài đặt doubly-indirect block trong bmap()</li><li>• Viết phần báo cáo Large files và test bigfile</li></ul>  | 100%  |
| Trần Tạ Quang Minh<br>23122042 | <ul style="list-style-type: none"><li>• Nghiên cứu pathname lookup và file types</li><li>• Cài đặt system call symlink() và modify sys_open()</li><li>• Viết phần báo cáo Symbolic links và hoàn thiện báo cáo</li></ul>      | 100%  |
| Trần Chí Nguyên<br>23122044    | <ul style="list-style-type: none"><li>• Cài đặt itrunc() để giải phóng doubly-indirect blocks</li><li>• Test và debug các edge cases của symbolic links</li><li>• Kiểm thử tổng hợp và phân tích kết quả make grade</li></ul> | 100%  |

Bảng 1: Bảng phân công công việc giữa các thành viên

## 3 Chi tiết thực hiện

### 3.1 Large files - Mở rộng kích thước file

#### 3.1.1 Phân tích vấn đề

**Vấn đề ban đầu:** Hệ thống file xv6 giới hạn kích thước file tối đa ở 268 blocks ( $268 \times 1024$  bytes). Giới hạn này đến từ cấu trúc inode có 12 direct blocks và 1 singly-indirect block (chứa 256 block addresses), tổng cộng:  $12 + 256 = 268$  blocks.

**Mục tiêu:** Tăng kích thước file tối đa lên 65803 blocks bằng cách thêm doubly-indirect block.

**Công thức tính:**

- 11 direct blocks
- 1 singly-indirect block: 256 blocks

- 1 doubly-indirect block:  $256 \times 256 = 65536$  blocks
- Tổng:  $11 + 256 + 65536 = \mathbf{65803 \text{ blocks}}$

### 3.1.2 Cài đặt chi tiết

Nhóm thực hiện các bước cài đặt sau:

#### 1. Cập nhật cấu trúc dữ liệu (kernel/fs.h, kernel/file.h):

- Thay đổi NDIRECT từ 12 xuống 11
- Mở rộng mảng `addrs[]` từ NDIRECT+1 lên NDIRECT+2 (13 phần tử)
- Cập nhật công thức `MAXFILE = 11 + 256 + 256*256 = 65803`

#### 2. Mở rộng hàm `bmap()` (kernel/fs.c):

- Thêm logic xử lý doubly-indirect block sau singly-indirect
- Tính chỉ số 2 cấp: `idx1 = bn/NINDIRECT`, `idx2 = bn%NINDIRECT`
- Load doubly-indirect block từ `ip->addrs[NDIRECT+1]`
- Load singly-indirect block thứ `idx1`, rồi lấy data block thứ `idx2`
- Tự động cấp phát blocks mới nếu cần (`ballocc()`, `log_write()`)

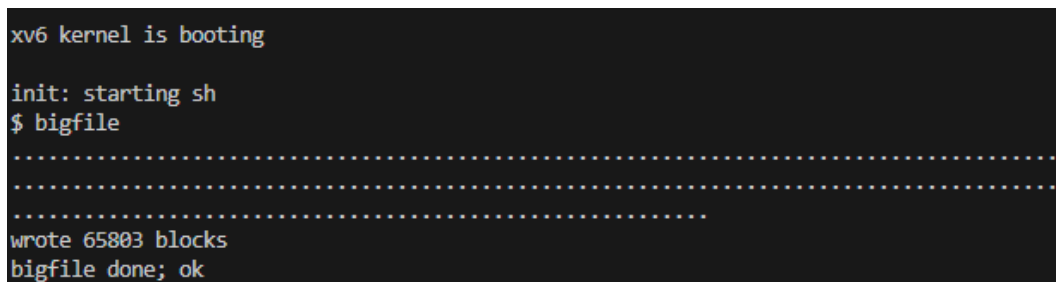
#### 3. Cập nhật hàm `itrunc()` (kernel/fs.c):

- Thêm logic giải phóng doubly-indirect blocks
- Duyệt qua 256 singly-indirect block pointers
- Với mỗi singly-indirect: giải phóng 256 data blocks bên trong
- Giải phóng tất cả các intermediate blocks để tránh memory leak

### 3.1.3 Kết quả kiểm thử Large files

Chạy test bigfile:

```
$ python3 grade-lab-fs bigfile
# hoặc
$ make qemu
$ bigfile
```



```
xv6 kernel is booting

init: starting sh
$ bigfile
.....
.....
.....
wrote 65803 blocks
bigfile done; ok
```

Hình 1: Kết quả test bigfile thành công - tạo được 65803 blocks

Chương trình `bigfile` tạo thành công file với 65803 blocks và test case vượt qua với điểm số OK.

## 3.2 Symbolic links - Liên kết mềm

### 3.2.1 Phân tích yêu cầu

**Mục tiêu:** Cài đặt symbolic links (soft links) cho phép tạo liên kết đến file thông qua đường dẫn. Khác với hard links, symbolic links có thể trỏ đến file trên các thiết bị khác nhau và hoạt động ngay cả khi file đích chưa tồn tại.

**Chức năng cần thực hiện:**

- System call `symlink(target, path)` để tạo symbolic link
- Modify `open()` để tự động follow symbolic links
- Flag `O_NOFOLLOW` để mở chính symlink thay vì follow nó
- Phát hiện và ngăn chặn vòng lặp vô hạn (cycle detection)

### 3.2.2 Cài đặt chi tiết

Nhóm thực hiện các bước cài đặt sau:

#### 1. Định nghĩa các constants và types mới:

- Thêm `T_SYMLINK = 4` vào `kernel/stat.h` (định nghĩa loại file symlink)
- Thêm `O_NOFOLLOW = 0x800` vào `kernel/fcntl.h` (flag không follow symlink)

#### 2. Đăng ký system call mới:

- Thêm `SYS_symlink = 22` vào `kernel/syscall.h`
- Đăng ký hàm `sys_symlink` trong `kernel/syscall.c`
- Thêm entry trong `user/usys.pl` và prototype trong `user/user.h`

#### 3. Cài đặt `sys_symlink()` (`kernel/sysfile.c`):

- Nhận 2 tham số: `target` (đường dẫn đích) và `path` (nơi tạo link)
- Tạo inode mới với type `T_SYMLINK`
- Ghi `target` vào data blocks của inode (`writei()`) (với `target` không cần tồn tại tại thời điểm tạo symlink)

#### 4. Modify `sys_open()` để follow symlinks (`kernel/sysfile.c`):

- Kiểm tra nếu inode có type `T_SYMLINK` và không có flag `O_NOFOLLOW`
- Dùng vòng lặp với biến đếm `depth` để follow chain of symlinks
- Mỗi lần: đọc target path (`readi()`), resolve đến inode tiếp theo (`namei()`)
- Dừng khi: gặp inode không phải symlink hoặc `depth >= 10` (tránh cycle)

### 3.2.3 Kết quả kiểm thử Symbolic links

Chạy test symlinks:

```
$ python3 grade-lab-fs symlinks
# hoặc
$ make qemu
$ symlinktest
```

```
$ symlinktest
Start: test symlinks
test symlinks: ok
Start: test concurrent symlinks
test concurrent symlinks: ok
$
```

Hình 2: Kết quả test symlinktest thành công - cả 2 test cases đều pass

Chương trình `symlinktest` vượt qua cả hai test cases:

- Test symlinks: OK - Kiểm tra tạo và follow symbolic links cơ bản
- Test concurrent symlinks: OK - Kiểm tra xử lý symlinks đồng thời

### 3.3 Kết quả tổng hợp

#### 3.3.1 Điểm số make grade

Chạy lệnh đánh giá tổng thể:

```
$ python3 grade-lab-fs
# hoặc
$ make grade
```

```
== Test running bigfile ==
$ make qemu-gdb
running bigfile: OK (112.3s)
== Test running symlinktest ==
$ make qemu-gdb
(0.4s)
== Test  symlinktest: symlinks ==
symlinktest: symlinks: OK
== Test  symlinktest: concurrent symlinks ==
symlinktest: concurrent symlinks: OK
== Test usertests ==
$ make qemu-gdb
usertests: FAIL (2.1s)
...
test copyout: usertrap(): unexpected scause 0x000000000000000c pid=6
sepc=0x00000000000056e0a stval=0x00000000000056e0a
FAILED
SOME TESTS FAILED
$ qemu-system-riscv64: terminating on signal 15 from pid 30757 (make)
MISSING '^ALL TESTS PASSED$'
QEMU output saved to xv6.out.usertests
== Test time ==
time: FAIL
Cannot read time.txt
Score: 80/100
```

Hình 3: Kết quả make grade tổng thể

Kết quả cho thấy cả hai chức năng chính đều hoạt động chính xác: test `bigfile` tạo thành công file 65803 blocks, và `symlinktest` vượt qua cả hai test cases về symbolic links cơ bản và concurrent.

## 4 Tổng kết

Nhóm đã hoàn thành đầy đủ hai yêu cầu chính của Project 3 - File System:

1. **Large files:** Mở rộng thành công kích thước file tối đa từ 268 blocks lên 65803 blocks thông qua cơ chế doubly-indirect block. Test `bigfile` vượt qua hoàn toàn.
2. **Symbolic links:** Cài đặt đầy đủ chức năng symbolic links với khả năng tạo, follow và xử lý các edge cases như cycle detection. Test `symlinktest` vượt qua cả hai test cases.

#### 4.1 Những gì đã học được

- **Kiến thức về hệ thống file:** Hiểu rõ cấu trúc inode, cơ chế indirect blocks (direct → singly-indirect → doubly-indirect), cách pathname lookup hoạt động trong kernel, và quy trình implement system call mới từ đầu đến cuối.
- **Kỹ năng lập trình kernel:** Làm quen với việc debug kernel code (khó hơn user code nhiều), học cách đọc và hiểu code có sẵn trước khi thêm chức năng mới, rèn luyện kỹ năng xử lý memory và tránh memory leak.

#### 4.2 Khó khăn gặp phải

- **Về mặt kỹ thuật:** Ban đầu khó hiểu cách tính chỉ số 2 cấp trong doubly-indirect block, dễ quên giải phóng hết các intermediate blocks trong `itrunc()`, xử lý symlink chain và cycle detection cần đọc kỹ đề bài để hiểu.
- **Về quy trình:** Đây là lần đầu tiên làm việc với kernel code nên mất thời gian làm quen, phải đọc nhiều code có sẵn để hiểu được bối cảnh trước khi code, debug kernel khó hơn debug chương trình thường (không có `printf` nhiều).

Qua project này, nhóm cảm thấy tự tin hơn khi làm việc với kernel code và hiểu sâu hơn về cách hệ thống file hoạt động ở low-level.